

e Secure

1. Simplify processes to comply with internal processes, controls and industry regulations without compromising speed

1. Run security scans earlier and for all projects

More Accessible

1. Allow anyone to be a creator

1. Make GitLab easier to use, higher System Usability Scale (SUS) score

1. Visibility into DevSecOps success

Cadence and Alignment

Our purpose is on a 30 year cadence

Mission

Our purpose informs our mission, and our mission is the way we realize our purpose. Our mission is to make it so everyone can contribute by using our product, to our product, and to our company.

By making it so everyone can contribute, we increase access for people to be creators. With more contributors and more creators, we increase both the volume and velocity of innovation. More innovation drives economic progress that benefits consumers, businesses, and the economy as a whole. As a result, innovation both directly and indirectly increases the total volume of available opportunities and average value of each individual opportunity.

Access to a broader set of more valuable opportunities ultimately increases people's lifetime earnings.

Our mission is on the same 30 year cadence as our purpose. Our purpose informs our mission, which directly or indirectly informs the rest of the items in our cadence. As a result, our purpose indirectly informs all of the items in our cadence, and progress for the items on our cadence page creates progress for both our mission and purpose.

Purpose Resources

Additional Purposes resources:

1. Creating a Meaningful Corporate Purpose

1. Find Your Why: A Practical Guide for Discovering Purpose for You and Your Team

{{% include "includes/take-gitlab-for-a-spin.md" %}}

SECTION: company/quote-to-cash.md

Overview

The Quote-to-Cash (O2C) process encompasses Customer Account Management, Order Fulfillment, Billing, and Accounts Receivable functions. The coordination of this process is owned by the Enterprise Applications team.

This is a cross-functional page which is meant to be the source of truth for our Quote to Cash (Q2C) systems and underlying data systems and processes.

An efficient Quote-to-Cash system makes purchasing, activating, and managing GitLab subscriptions as easy as possible.

- Improves customer satisfaction, and also streamlines our go-to-market (GTM) processes and accelerates revenue growth for the company.
- Ensures consistency between the data for some of the most critical business objects/entities across all the Source Systems.
- Master Data Objects for all core business critical objects will have same definitions and data across all the systems.
- Proper alignment/sync exists between all the data source systems - Zuora, CustomerDot, SFDC and GitLab.com along with 1:1 mapping between Zuora Billing Object Model and CustomersDot.
- Improves data quality and ensures a single lineage for understanding the paid namespace conversion journey, the first paid subscription for a given namespace, free/trial to paid conversion analysis etc.

Teams

Quote-to-Cash system projects and initiatives often require close collaboration across functions and teams. The teams most often involved include: Enterprise Applications, Billing Ops, Fulfillment, Field Operations, Support, and Data.

Systems

The Q2C systems consists of several systems including Salesforce, Zuora (CPQ, 360, Billing, Revenue), CustomersDot, and NetSuite.

Salesforce

- Salesforce is used as our CRM tool for managing customer Leads, Contacts, Accounts, Opportunities and Quotes.
- Salesforce is owned by the Enterprise Applications team at GitLab, and implements changes from the process owners.
- The Quoting process itself is owned by the Deal Desk team.

Zuora CPQ

- Zuora CPQ is our Configure, Price, Quote tool used for Sales Assisted deals
- Zuora CPQ is a managed package in Salesforce that has been extended for Quote Approvals by Enterprise Applications
- The Quote Approval process itself is owned by the Deal Desk team.

Zuora 360

- Zuora 360 is a stock connector between Zuora and Salesforce, it transfers Zuora subscription information to Salesforce.
- Zuora 360 the job is owned by Enterprise Applications, the extension of Zuora Subscription data in Salesforce for Add-ons and Renewals deal is also owned by Enterprise Applications.
- The Add-On and Renewal processes are owned by the Deal Desk team.

Zuora Billing

- Zuora is used as our billing and revenue tool for managing customer subscriptions, payments and invoicing.
- Zuora is owned by the Enterprise Applications team at GitLab.
- The Billing process itself is owned by the Billing Operations team

Zuora Revenue

- Zuora Revenue is our automated revenue recognition application that meets current and future U.S. GAAP, including the new ASC 606 and IFRS 15 revenue standards.

CustomersDot (Customers Portal)

- CustomersDot is used when the customer logs in to manage their subscriptions
- GitLab engineers created CustomersDot and it is owned by the Fulfillment team
- CustomersDot integrates with Zuora to enable self-service purchasing and subscription management

NetSuite

- NetSuite is the company Enterprise Resource Planning (ERP) system, which is primarily managed by the Finance team.
- The platform allows enhanced dimensional reporting as well as multi-currency and multi-entity reporting. This is where the General Ledger resides and all financial activity is ultimately recorded, which is critical to reporting the financial health of the company.

Architecture

Data Objects

Equivalent Data Objects

This table shows the equivalent data objects across systems:

GitLab	CustomersDot	Zuora	SFDC
Organization	BillingAccount	Account	BillingAccount
User	User	Contact	Contact
-	Order	Order	Opportunity & primary quote
-	Subscription	Subscription	Subscription
License	License	-	-

Note: In SFDC, a SFDC BillingAccount is not the same as a SFDC Account. A SFDC Account can have many BillingAccounts.

Note: The Order object in CustomersDot is not the same as the Order object in Zuora, they have different definitions. Orders in CustomerDot are more like subscriptions in Zuora than they are Orders in Zuora. More architecture and definition work needs to be done on the Order object in CustomersDot.

CustomerDot Object Model

The following is a focused view of the database ERD for the newly proposed data architecture.

mermaid
erDiagram

```

CustomersDotUser ||--o{ CustomersDotBillingAccountMembership : "has many"
CustomersDotUser ||--o{ ZuoraContact : "has many"
CustomersDotBillingAccount ||--o{ CustomersDotBillingAccountMembership : "has many"
CustomersDotBillingAccount ||--o{ CustomersDotCloudActivation : "has many"
CustomersDotBillingAccount ||--o{ CustomersDotLicense : "has many"
CustomersDotBillingAccount ||--o{ CustomersDotOrder : "has many"
CustomersDotBillingAccount ||--|| GitLabOrganization : "has one"
CustomersDotCloudActivation ||--o{ GitLabCloudActivation : "has many"
CustomersDotLicense ||--o{ GitLabLicense : "has many"
CustomersDotUser ||--|| GitLabUser : "has one"
CustomersDotOrder ||--o{ CustomersDotSubscription : "has many"
CustomersDotSubscription ||--|| ZuoraSubscription : "has one"
GitLabUser ||--o{ GitLabMember : "has many"
GitLabOrganization ||--o{ GitLabMember : "has many"
GitLabOrganization ||--o{ GitLabLicense : "has many"
GitLabOrganization ||--o{ GitLabCloudActivation : "has many"
ZuoraAccount ||--o{ ZuoraContact : "has many"
ZuoraAccount ||--o{ ZuoraPaymentMethod : "has many"
ZuoraAccount ||--o{ ZuoraOrder : "has many"
ZuoraAccount ||--o{ ZuoraSubscription : "has many"
ZuoraOrder ||--o{ ZuoraSubscription : "has many"
ZuoraAccount ||--|| CustomersDotBillingAccount : "has one"
SFDCAccount ||--o{ ZuoraAccount : "has many"
SFDCAccount ||--o{ CustomersDotBillingAccount : "has many"

```

```

CustomersDotBillingAccount {
    bigint id
    string zuoraaccountid
    string zuoraaccountname
    string zuoraaccountnumber
    string zuoraaccountentity
    string zuoraaccountvatid
    string salesforceaccountid
    timestamp createdat
}

```

```

    timestamp updatedat
}
CustomersDotBillingAccountMembership {
    bigint id
    bigint userid
    bigint billingaccountid
    timestamp createdat
    timestamp updatedat
}
CustomersDotUser {
    int id
    string firstname
    string lastname
    datetime createdat
    datetime updatedat
    string email
    string encryptedpassword
    string resetpasswordtoken
    datetime resetpasswordsentat
    datetime remembercreatedat
    integer signincount
    datetime currentsigninat
    datetime lastsigninat
    inet currentsigninip
    inet lastsigninip
    string provider
    string uid
    string country
    string state
    string city
    string zipcode
    string vatcode
    string company
    boolean billable
    string accesstoken
    string confirmationtoken
    datetime confirmedat
    datetime confirmationsentat
    string unconfirmedemail
    string address1
    string address2
    string companysize
    string authenticationtoken
    string phonenumber
    boolean loginactivated
    string refreshtoken
    datetimewithtimezone accesstokenexpiresat
    datetimewithtimezone tokenrefreshlastattemptedat
}
CustomersDotCloudActivation {

```

```
    bigint id
    bigint billingaccountid
    string activationcode
    string subscriptionname
    boolean supersonicsaware
    datetime seatutilizationremindersentat
    timestamp createdat
    timestamp updatedat
}
```

```
CustomersDotLicense {
    bigint id
    bigint billingaccountid
    uuid licensefilemd5
    bigint creatorid
    datetimewithtimezone createdat
    datetimewithtimezone updatedat
    datetimewithtimezone lastsyncedat
    datetimewithtimezone nextsyncat
    integer userscount
    integer previoususerscount
    integer trueupquantity
    date expiresat
    date startsat
    date trueupfrom
    date trueupto
    boolean trial
    boolean cloudlicensingenabled
    string plancode
    string planname
    string zuorasubscriptionid
    string email
    string name
    string company
    string zuorasubscriptionname
    text notes
    text licensefile
    datetimewithtimezone activatedat
    boolean autorenewenabled
    boolean seatreconciliationenabled
    boolean operationalmetricsenabled
    boolean reconciliationcompleted
    boolean offlinecloudlicensingenabled
}
```

```
CustomersDotOrder {
    int id
    bigint billingaccountid
    string zuoraproductrateplanid
    string zuorasubscriptionid
    string zuorasubscriptionname
    date startdate
}
```

```

date enddate
integer quantity
datetime createdat
datetime updatedat
string glnamespaceid
string glnamespacename
string amendmenttype
boolean trial
datetime lastextraciminutessyncat
string zuoraaccountid
datetime increasedbillingratenotifiedat
boolean reconciliationaccepted
integer trialexensiontype
string source
datetimewithtimezone seatoveragenotifiedat
datetimewithtimezone autorenewerrornotifiedat
}
CustomersDotSubscription {
}
ZuoraAccount {
  string crmid
}
ZuoraContact {
}
ZuoraSubscription {
}
ZuoraPaymentMethod {
}
SFDCAccount {
}
GitLabLicense {
}
GitLabCloudActivation {
}
GitLabOrganization {
}
GitLabMember {
}
GitLabUser {
}

```

Zuora Billing Object Model

Zuora provides a diagram of the relationships of Zuora's Billing Object Model

To reduce the amount of data issues across our systems, our goal is to try to ensure we have a 1:1 mapping between Zuora Billing Object Model and CustomersDot.

Zuora and Salesforce Integrated Object Model

Zuora CPQ is used to connect Zuora with Salesforce.

Billing Account Master Data Object

Billing Account is a core business entity that holds a paying customer's most critical account information such as their contact details, payment terms, and payment methods. And this information is used to track subscriptions, amendments, and transactions, such as invoices and payments. The data from this Master Data Object is actively being used in several GTM, Product Usage and Data science Propensity models to understand the billing information of paid customers better etc..

First, we focussed on improving the data architecture alignment between Zuora Billing Accounts and CDot Billing Accounts in order to proceed with developing a Conformed Dimension for Billing Account Entity.

The Customer model in CDot has known design flaws as it represents a combination of both a Contact (individual user) and an Account (organization). Research for this epic can be found in this Spike issue.

Projects

The following are architecture projects undertaken by Quote to Cash teams to better align systems.

Fulfillment data architecture plan

The Fulfillment Team is re-architecting our Quote 2 Cash Systems, in particular CustomersDot, in a way that promotes more reliability, sustainability, and flexibility. This handbook page serves as the SSOT for the Q2C systems with links back to functional department pages and artifacts.

- 1. Fulfillment SSOT Plan: dataarchitecture
- 1. Central Data Team SSOT Plan: dataarchitecture
- 1. Sales Systems SSOT Plan: To be added
- 1. Enterprise Apps SSOT Plan: dataarchitecture

Zuora serves as the source of truth for Zuora Account and Zuora Contact data once a Subscription is purchased. Prior to a purchase, a user can register for CDot which creates a CustomersDot User record that isn't associated with an CustomersDot BillingAccount (because it doesn't exist yet). Once purchased, the CustomersDot BillingAccount record is created along with the related CustomersDot BillingAccountMembership.

Given that CustomersDot User/Zuora Contact and CustomersDot BillingAccount/Zuora Account information can be edited by users directly in CDot or directly in Zuora (or indirectly via SFDC), we need to be mindful of syncing this data between CDot and Zuora. In particular, if we can't use Zuora callouts to keep the CustomersDot BillingAccount and CustomersDot User records

in sync, we will explore Zuora Custom Events.

A CustomersDot User record in CDot is tied to one email address. This email address can be associated with multiple Zuora Accounts, and therefore have multiple Zuora Contacts. Each of these Zuora Contacts could be modified independently. For instance, a billing admin may choose to change the address for Contact A for the billing entity in the US, but not choose to change the address for Contact B (associated with the same email address) for the billing entity in Europe. For this reason, contact metadata could eventually be stored on the CustomersDot BillingAccountMembership model, but we are choosing to keep this lightweight to begin with to reduce scope. We will start by fetching this data from Zuora.

CustomersDot BillingAccount management

Introduce CustomersDot BillingAccount

- Improve Bill To / Sold To Contact Management in CustomersDot Admin - Completed
- Iteration 1A: Align CustomersDot BillingAccounts and Users to Zuora objects - Completed
- Iteration 1B: A single CustomersDot BillingAccount can have multiple CustomersDot Users - 90% Complete
- Iteration 1C : A single CustomersDot User can have multiple BillingAccounts - Not Started
- Iteration 1D: Clean up legacy data objects - Not Started

Background

In this epic, the focus is on improving the data architecture of CustomersDot to better align with Zuora Billing Accounts. The Customer model in CDot has known design flaws as it represents a combination of both a Zuora Contact (individual user) and a Zuora Account (organization).

Problem

Currently CustomersDot (CDot) has a data object for Customer (e.g. customers DB table) which serves a few different functions:

- A user (with a unique email address) that can login CustomersDot
- Contact information related to a physical person within a Company with metadata like first and last name, email, mailing address, etc.
- Company information that is associated with a Zuora Account with company name.

It is important to note that a Zuora Account maps to a company/customer account which can have many users or contacts. It should not map to one user in particular. In the current architecture, a zuoraaccountid can be shared with more than one Customer but this isn't ideal. We need to have an architecture that accurately reflects the data structure from Zuora and our business model. In CustomersDot, we need to have a data architecture that accurately reflects Zuora's Billing Objects Model.

Examples of issues this could benefit

1. CDot Issue 3626
1. GL Issue 332236
1. Epic 2160

1. CDot Issue 242
1. CDot Issue 695
1. GL Issue 338546

Plan

We recommend adding a few new models to the CustomersDot data architecture to better reflect the Zuora data structure.

Most importantly, CDot lacks a model for the CustomersDot BillingAccount (aka. "Company" / "Billing Entity"). Adding this model will allow us to separate the data currently in the CustomersDot Customer model that belongs at a BillingAccount level (see below for an example). We will also be renaming CustomersDot Customer model to CustomersDot User as the current naming is a bit confusing due to Salesforce having a different definition for what a Salesforce Customer is.

- zuoraaccountid
- companyname
- vatcode
- salesforceaccountid

Additionally, we can add a join table (e.g. billingaccountmemberships table) between the CustomersDot BillingAccount and CustomersDot User models which will allow a CustomersDot BillingAccount to have multiple CustomersDot Users. Likewise, a CustomersDot User could be associated to many CustomersDot BillingAccounts.

With this data structure, CDot should consider the data structure of CustomersDot BillingAccount and CustomersDot User as a way of determining which users of CDot should have access to view / manage subscriptions for a Zuora Account. CDot could rely on Zuora for providing the source of truth for who the contacts are for the Zuora Account and likely their metadata (e.g. name, address).

Currently, CDot creates a CustomersDot User record for the SoldTo contact associated with the Subscription Account. In this epic, we could also create a CustomersDot User record for the BillTo contact as well for the same Subscription Account, but this isn't a requirement. This will effectively allow both types of contacts to login to CDot and manage the Account's subscriptions.

Why do we need the ability to associate a CustomersDot User with many CustomersDot BillingAccounts?

We considered only allowing a CustomersDot User to be associated with a single CustomersDot BillingAccount, however, we determined that we should build a many-to-many mapping capability because:

1. Zuora currently allows for a single user to map to multiple billing accounts.
1. We see examples in production where this is already happening (over 1800 user accounts in Zuora currently map to multiple Billing Accounts).

Enabling this will bring parity between CustomersDot and Zuora.

Update: CDot has added BillingAccount, BillingAccountMembership and BillingAccountContact models to align with Zuora Billing models.

Source of Truth

Zuora serves as the source of truth for Zuora Account and Zuora Contact data once a Subscription is purchased. Prior to a purchase, a user can register for CDot which creates a CustomersDot User record that isn't associated with an CustomersDot BillingAccount (because it doesn't exist yet). Once purchased, the CustomersDot BillingAccount record is created along with the related CustomersDot BillingAccountMembership.

Given that CustomersDot User/Zuora Contact and CustomersDot BillingAccount/Zuora Account information can be edited by users directly in CDot or directly in Zuora (or indirectly via SFDC), we need to be mindful of syncing this data between CDot and Zuora. In particular, if we can't use Zuora callouts to keep the CustomersDot BillingAccount and CustomersDot User records in sync, we will explore Zuora Custom Events.

A CustomersDot User record in CDot is tied to one email address. This email address can be associated with multiple Zuora Accounts, and therefore have multiple Zuora Contacts. Each of these Zuora Contacts could be modified independently. For instance, a billing admin may choose to change the address for Contact A for the billing entity in the US, but not choose to change the address for Contact B (associated with the same email address) for the billing entity in Europe. For this reason, contact metadata could eventually be stored on the CustomersDot BillingAccountMembership model, but we are choosing to keep this lightweight to begin with to reduce scope. We will start by fetching this data from Zuora.

GitLab.com SSO as the only sign in option for CustomersDot

With GitLab.com SSO as the only sign in option for Customers Portal, we can eliminate the legacy sign up (email and password) and prepare CDot for future integration with a third party e-commerce provider.

Our goal is to streamline the experience between SaaS/self-managed and web direct/sales-assisted customers in how they access their subscription information.

Overall, this will lead to a more secure environment for our customers in CDot and allows us to establish a 1:1 relationship between a CDot customer and a GitLab.com account.

GitLab.com SSO as the only sign up option for CustomersDot

- CDot SSO: Remove email and password sign up
- CDot SSO: Enhance experience for first time login to CDot
- CDot SSO: Migrate more CDot customers to use the GitLab SSO for login

Align CustomersDot Orders with Zuora Orders

This work focuses on breaking apart the CustomersDot Order table and moving towards a data structure that is more representative of the Zuora Subscriptions table.

See the Architecture Blueprint for full details.

Align CustomersDot Orders to Zuora objects

- Phase 1: Implement Zuora Cache Models
- Phase 2: Implement Zuora Cache Sync and Backfill
- Phase 3: Utilize Zuora Cache Models
- Phase 4: Replace CDot Order with Subscription

Snowflake Data Warehouse and dbt (data build tool)

We extract data from the Quote to Cash systems towards Snowflake and use dbt to transform the data into data models for reporting and analysis.

Data Architecture Plan

The Fulfillment Team is re-architecting our Quote 2 Cash Systems, in particular CustomersDot, in a way that promotes more reliability, sustainability, and flexibility. A significant result of the new architecture will be to have the same customer definition in CustomerDot, Zuora, and Salesforce. This definition is based on the Billing Account in Zuora and the key for the customer is the billingaccountid. It is necessary to re-architect the data model in Snowflake and dbt in order to align the customer definition to CustomersDot data tables. The data models built that model the Zuora and Salesforce systems have the correct customer definition and no rearchitecting needs to happen for those models at this time.

Important Callouts for the Customer Definition in the Unified Q2C data model:

1. The unified customer key is the BillingAccountId which joins the systems together. The BillingAccountId appears on Q2C systems as follows:

1. Zuora.Account.AccountId

1. CustomersDot.BillingAccounts.BillingAccountId

1. Salesforce.BillingAccount.BillingAccountId

1. DRAFT: GitLabDotcom.Organization.BillingAccountId The Organization Entity Object has not been created yet in GitLabDotcom and is still in the validation process. The Billing Account Id Foreign key needs to be determined.

1. The source id data (where the data is originated and is considered the SSOT) for billingaccountid lives in Zuora.

1. The customer definition is only unified for billing accounts that are in Zuora and Salesforce which are generally limited to paying and formerly paying customers and edu/oss customers.

1. The target state for trials is to put them into Zuora. At that time, we would be able to have a unified definition for customers across CustomersDot, Salesforce, and Zuora for their trial to paid activity.

1. The target state for how to unify the customer definition for free customers across the Q2C systems needs to be determined. The quantity of free users presents challenges for putting them into Salesforce and Zuora. This area needs further exploration.

1. The Account object in Salesforce can have customers that will have a billing account or it can have prospect accounts that do not have a billing account.

The Q2C Re-Architecture will be implemented in various phases with a clear communication to the audience and other cross-functional teams on the Expected Outcomes and Deliverables from each phase.

Conformed Dimensions

Conformed dimensions allow facts and measures to be categorized and described in the same way across multiple facts and/or data marts, ensuring consistent reporting across the enterprise.

Conformed Dimensions correspond to Master data and are critical building blocks of any Data Warehousing Environment.

The below Master Data objects for core business entities will be created that can be used in the different applications across the organization, along with their associated metadata, attributes, definitions, roles, connections, and taxonomies.

- Billing Account
- Customer/User
- Organization
- Order
- Lead
- License

The Master Data Objects development work in Snowflake/dbt is tracked in these two Epics: Billing Account, Orders, Ramps and Customers, Contacts/Users, Leads.

Below is the Entity Relationship Diagram for the Re-architected data model in Snowflake. The Target State tab shows how the business entities we extract from the CustomersDot, Zuora, Salesforce, and GitLab.com source systems connect with each other.

```
<div style="width: 640px; height: 480px; margin: 10px; position: relative;"><iframe
allowfullscreen frameborder="0" style="width:640px; height:480px"
src="https://lucid.app/documents/embedded/c8f1520c-e59b-4551-a9db-bfce88bb84dc"
id="0GkOGAjoDO."></iframe></div>
```

Conformed Dimension Design for Core Business Objects

```
<details markdown=1>
```

```
<summary><b> Billing Account Master Data Object/Entity (Conformed Dimension Design)
</b></summary>
```

```
<div style="width: 640px; height: 480px; margin: 10px; position: relative;"><iframe
allowfullscreen frameborder="0" style="width:640px; height:480px"
src="https://lucid.app/documents/embedded/ce1cb165-8cae-4e91-9d15-34fd59148f8c"
id="f7t5DXEuu7L"></iframe></div>
```

```
</details>
```

```
<details markdown=1>
```

```
<summary><b> Order Master Data Object/Entity (Conformed Dimension Design) </b></summary>
```

```
<div style="width: 640px; height: 480px; margin: 10px; position: relative;"><iframe
```

allowfullscreen frameborder="0" style="width:640px; height:480px"
src="https://lucid.app/documents/embedded/a7a3b55a-c33c-4806-b79b-0a0f28917735"
id="75t5oZ6PBayn"></iframe></div>

</details>

<details markdown=1>

<summary> Customer/User Master Data Object/Entity (Conformed Dimension Design)
</summary>

<div style="width: 640px; height: 480px; margin: 10px; position: relative;"><iframe
allowfullscreen frameborder="0" style="width:640px; height:480px"
src="https://lucid.app/documents/embedded/0e46bd1c-0d15-40d4-8755-d2b45bcc4346"
id="L6t5L7s2wK5-"></iframe></div>

</details>

SECTION: company/stewardship.md

Business Model

GitLab Inc. is a for profit company that balances the need to improve the open source code of GitLab with the need to add source-available features in order to generate income. We have an open core business model and generate almost all our revenue with subscriptions to paid tiers. We recognize that we need to balance the need to generate income and with the needs of the open source project.

We have tried different business models, and many didn't work. As a company, we realized we needed recurring revenue to continue our mission, and we introduced source available code that is proprietary. Initially there was a worry we would stop working on the open source code but the community saw we were able to accelerate the work on open-source code too.

Promises

We promise that:

1. When a feature is open source we won't move that feature to a paid tier. Features might be removed from the open source codebase in other cases, for example when combining features from multiple tiers into one new feature. To be clear, this promise only applies to open sourced features, features in paid tiers might move to a higher tier.
1. We won't introduce features into the open source codebase with a fixed delay, if a feature is planned to land in both it will be released simultaneously in both.
1. We will always release and open source all tests that we have for a open source feature.
1. The open source codebase will have all the features that are essential to running a large 'forge' with public and private repositories.
1. The open source codebase will not contain any artificial limits (repositories, users, size,

performance, requiring a trademarked header, etc.).

1. All stages of the DevOps lifecycle (plan, create, verify, package, release, configure, monitor) will have some open source features.

1. The majority of new features made by GitLab Inc. will be open source.

1. The product will be available for install and/or download in the menu of the website.

1. You can install it without you having to submit an email address or sign in.

1. We will always allow you to benchmark the performance of GitLab.

1. We will make our Free tier easily discoverable.

1. We will always make it clear what is proprietary and what is open source code. This will be implemented with a separate directory for EE code and a Git repo downstream that is open source only.

Software as a Service

Our stewardship promise applies only to the GitLab open source codebase, not to GitLab, Inc services such as GitLab Software as a Service (SaaS).

For GitLab SaaS, we may add limits around usage (e.g., storage, compute, traffic) or proxies for usage (e.g., number of users in a group) as we seek to optimize costs and revenues.

Our stewardship promise allows other companies and organizations to provide a SaaS offering based on the GitLab open source codebase.

What Features Are Paid-Only

To determine what is open source and what not we ask ourselves: Who cares the most about the feature.

If the likely buyer is an individual contributor the feature will be open source, otherwise it will be source-available (proprietary).

There aren't any features that are only useful to managers, directors, and executives.

So for every source-available feature there will be an individual contributor who cares about it.

We're not saying that there aren't any individual contributors that care about the feature, just that we think that other buyers are relatively more likely to care about it.

The more of GitLab that you use the more likely it is that you benefit from a higher tier.

Even a single person using GitLab might be best off using our highest tier.

It is hard to get the tier right, and if we put something in a tier that is too high we won't hesitate to open-source it or move it to a lower tier. We listen to our community in order to find what we feel is the right balance, and we iterate and make changes based on their feedback. At the same time, the premium product needs to hold value, and we believe we provide that.

All stages of the DevOps lifecycle have at least some features available in GitLab Core. There are companies using GitLab Core with more than 10,000 users.

If people ask us why a certain feature is paid we might reply with a link to this section of the handbook. We do not mean to imply you don't need the feature. Feel free to make the argument for moving it to another tier, we're listening.

If the wider community contributes a feature that is currently source-available we use the

process linked in
Contributing an existing feature to open-source it.

Why Release Simultaneously In Both

Sometimes people suggest having features in EE for a limited time.
An example of a limited time release strategy is the Business Source License that keeps features proprietary for 3 years.

At GitLab we want to give everyone access to most of the features (and all the essential ones) at the date they are announced.

We want to give people the option to both run and contribute to an open source edition that is maintained and that includes the most recent security fixes.

From time to time we do open source a feature that was previously proprietary.
We do this when we realize we made a mistake applying our criteria, for example when we learned that a branded homepage was an essential feature or when we brought GitLab Pages to the Community Edition.

Our plan is to become the most popular tool for people's own git hosting service; we've managed that so far. Secondly, we want to get to be the one with the most revenue. Thirdly, we want to become the most popular tool for hosting private repos. Once we've reached that, we want to be the most popular tool for hosting public repos. And, lastly, we want to be the number one tool for people to host not just code but books, tech papers, visual models, movies, etc. More info on this is on our strategy page.

How Open Source Benefits From Open Core

GitLab Inc. has an open core business model that includes source-available code and selling subscriptions.

This benefits the open source part of GitLab in the following ways:

1. New features being made by GitLab Inc. that are open source
1. Responsible disclosure process and security fixes
1. Release management including a monthly release and patches
1. Packaging GitLab in our Omnibus packages
1. Running a packages server
1. Dependency upgrades (Rails, gems, etc.)
1. Performance improvements

Common Criticisms Of Open Core

There are valid criticisms of Open Core.

At Open Core Summit in 2019, Deb Bryant of Red Hat highlighted the following four:

Participation Is Constrained.

Open Core companies may limit participation to keep premium features from being contributed. -
We actively work to grow participation since our mission is that Everyone can Contribute.
Also, we do not say no by-default to having existing paid features contributed to our open source project.

Vendor Lock-In.

Premium features make it more difficult to switch workflows. -

GitLab the product plays well with others. As we outline,

> Many other applications integrate with GitLab, and we are open to adding new integrations to our technology partners page. New integrations with GitLab can vary in richness and complexity; from a simple webhook, and all the way to a Project Service.

> GitLab welcomes and supports new integrations to be created to extend collaborations with other products. GitLab plays well with others by providing APIs for nearly anything you can do within GitLab. GitLab can be a provider of authentication for external applications. GitLab is open source so people are very welcome to add anything that they are missing.

Community Is Devalued.

Community is seen as a marketing tool instead of as participants working to make the product better together. -

We care deeply about our community and depend on all GitLabbers to help us improve our category and stage maturity.

We have Merge Request Coaches who help contributors get their merge requests to meet the contribution acceptance criteria,

and wider community contributions per release is a GitLab KPI.

Deployment Constrained.

Customers are afraid of confusing or unsanctioned licenses entering a business environment. -

We don't use a limiting license such as SSPL. As we highlight in our docs,

GitLab Community Edition (CE) is licensed under the terms of the MIT License, which is an official Open Source license as defined by the Open Source Initiative.

Existing Contributed Open Source Features Will Not Become Source-Available

We never move existing features already in GitLab FOSS into a paid tier.

This applies regardless of whether the feature was created by GitLab team members or the wider community of contributors.

On occasion, the reverse does happen where we open source a previously source-available feature.

On occasion, we might change the product in a way that bundles functionality together or refactor functionality that was in different tiers.

Sometimes the new bundle will go to the Core/Free tier and sometimes to the paid tiers.

Contributing An Existing Feature To Open-Source It

When someone contributes an existing source-available feature to open-source code base, we have a hard

decision to make. We encourage contributors to focus on new features not already existing, so that both codebases of GitLab benefit from the feature, and we can avoid any difficult decisions.

We encourage contributors visit our direction page

to see features that are welcomed and also review CONTRIBUTING.md.

When someone contributes an existing feature to open-source it, we weigh a number of factors to

decide in accepting it, including:

1. What is the quality of the submitted code?
1. Is it a complete replacement of the source-available functionality?
1. Does it meet the contribution acceptance criteria?
1. Is it more relevant for mid-market organizations or larger?
1. Is the person or organization submitting this using GitLab in an SMB?
1. Did the person or organization submitting this contribute to GitLab before?
1. Is it something that many of our existing customers chose our paid tiers for?
1. Is it relevant for running a large open source forge?
1. Is it an original work or based on the source-available code?
1. Is there an actively maintained fork that incorporates this?
1. How many organizations are using this code in production?
1. How frequently has this functionality been requested and by whom?

We'll weigh all factors and you can judge our stewardship of the open source codebase based on the outcome. As of Dec 4, 2018, we had only two cases: One had low code quality and the other one copied the source-available code down to the last space. If you find these or other examples please link them here so people can get an idea of the outcome.

Contributing A Not-Yet-Existing Feature

When someone contributes a not yet existing feature on the issue tracker that has a paid tier label, and it has met the contribution acceptance criteria, we will accept it under whatever license (open-source or source-available) they prefer, provided that GitLab Inc. has not already started working on the feature. (The contribution should not contain any already existing source-available features in it.) We encourage contributors to @-mention the relevant product manager earlier in the development process (in the issue or merge request) to ensure GitLab team-members are not already working on the feature in order to avoid conflicts.

How We Think About Contributing More To Open Source

Some companies try to support "20% time" for contributing to open source projects or other self-directed work. While this has been discussed several times within GitLab, we intentionally do not implement something like this for the following reasons:

1. It is really 120% time.
1. It increases context switching and inadvertently leads to a decline in throughput as people split focus across disparate projects with different goals. The more work in progress at any one point in time, the longer it will take to drive any single item to completion. The end result is less open source over time.

Instead of optimizing locally across many different projects, we are able to create much more value and impact for the open source community by singularly focusing on our vision. Doing so creates a virtuous cycle that enables us to:

Make More Improvements Upstream.

GitLab is open core built entirely with open source. We rely on thousands of dependencies and

always try to contribute our improvements upstream.

Build More Open Source As Part Of GitLab.

Shipping as much open source as possible is critical to our business model because it increases the velocity of our flywheels. The faster they spin, the more we can contribute back.

Contribute At Scale.

GitLab is a single application for the entire DevOps lifecycle. By freely providing the top tier of GitLab to open source projects, we are enabling them to be more efficient, secure, and productive.

The combination of the above leads to global optimization on a much larger scale than anything we could contribute with a small portion of our time working on things that don't directly move our vision forward.

```
{{% include "includes/take-gitlab-for-a-spin.md" %}}
```

SECTION: company/still-a-startup.md

Still a Startup Overview

In the "Letter From Our CEO" in GitLab's S-1 (filed with the Securities and Exchange Commission on September 17, 2021), GitLab's then CEO, Sid Sijbrandij, documented some of the ways that GitLab plans to avoid the stagnation experienced by most early stage companies as they mature. This was the inspiration for this page.

A senior candidate once referenced the letter and said, "your letter is good at reminding everyone that you are still a startup and need to retain that mindset. It's super hard, as you know. When I reflect upon all the amazing things I experienced at [Company X], it is a poorly managed company. The systems (legal, procurement, security) that grow with success are also designed to manage the downside. I see the same thing here at [Company Y]. I'm fighting to re-create the challenger mindset to reflect our market position outside of virtualization and networking. Cruft everywhere. You have an opportunity to minimize this as you scale."

Most companies regress to the mean and slow down over time. We plan to maintain our startup ethos by doing the following:

Reinforcing our values

We have more than 20 documented ways to reinforce GitLab's values. Since hiring, bonuses, and promotions provide strong signals of what is valued and rewarded, we make values the lens through which we evaluate team member fit and advancement.

Quick and informed decisions

We are able to combine the advantages of consensus organizations and hierarchical organizations by splitting decisions into two phases. In the data gathering phase, we employ the best of consensus organizations as we encourage people to contribute their ideas and opinions. In the

decision phase, we benefit from the best of hierarchical organizations with one person, the directly responsible individual, deciding what to do without having to convince the people who made suggestions.

A directly responsible individual (DRI)

A DRI is a single person who owns decision making authority and responsibility for the success of a given workstream or initiative. We avoid confusion and empower team members by being clear about the DRI. With a few documented exceptions, the person who does the work resulting from the decision gets to make the decision. DRIs tend to have the context required for good decision making and are empowered by their ability to use their own judgement in doing what is best for the business.

Organize informal communications

Informal team member communications, such as a chat about life outside of work, are necessary for building trust. Trust is essential for great business results. Many businesses invest heavily in offices and facilities, because they believe offices are necessary for informal communication.

During the pandemic, many businesses that were forced to work remotely discovered that productivity increased. Many of these same businesses are now making plans to return to the office. One reason given is that not everyone can work from home. We solve this by allowing people to rent work space. The other main reason given is that people miss working from a central office with co-workers. I don't think that people miss the commute or the office furniture. They miss informal communication. Central offices are a really expensive, inconvenient, and indirect way to facilitate information communication. It is more efficient to directly organize informal communication.

For example, every person who joins GitLab has to schedule at least 5 coffee chats during their onboarding. We also have social calls, Ask Me Anything meetings with senior leaders, and 15 other explicit ways to encourage employee connections and relationship building. Intentionally organizing informal communication enables the trust-building conversations that are essential for collaboration. This can be more effective than relying on chance encounters in an office building. You can connect with team members throughout the world and across departments through a coffee chat. You may not meet people outside of your own floor in an office setting.

Challenge conventions

We do not do things differently for the sake of being different, and we use boring solutions whenever possible. That said, we're also willing to deviate from conventions when it can benefit GitLab and the wider community. Before the COVID-19 pandemic, GitLab was the largest all-remote company in the world. We now teach others how to succeed as remote companies and employees. We aim to be the most transparent company of our size. This transparency has had demonstrable benefits ranging from increased team member productivity to enhanced brand awareness. What some saw as a liability, we have shown to be a strength.

Bias for action

Decisions should be thoughtful, but delivering fast results requires the fearless acceptance of

occasionally making mistakes. Our bias for action may result in the occasional mistake, but it also allows us to course correct quickly. We keep the stakes low for mistakes for the sake of transparency. When people are comfortable communicating missteps, risk aversion and secrecy don't become the norm.

Not a family

Some companies talk about being a 'Family.' We don't think that is the right perspective. At GitLab, the relationship is not the end goal. The goal is results. We are clear about accountability and hold people to a clearly articulated standard. When people do not perform, we try to help them improve. If they still can't meet expectations, we let them go.

Time based release

We have introduced a new, enhanced version of our software every month for over nine years. A time based release ensures that when a feature is ready, its release will not be held up by another that is not. Aligned with our value of iteration, we try to reduce the scope of each feature so that it fits in a single release.

Individual innovation

We empower individuals to innovate. For example, we have designated coaches who support contributors from the wider community in getting their contributions to the point where they can be merged by GitLab. We also have an incubation department dedicated to quickly turning ideas into viable features and products.

Dogfooding

The best way to quickly improve GitLab is to use it ourselves, or dogfood it, so that we have a quick feedback loop. We use our own product even when a feature is in its early stages of development. This helps us to develop empathy with users and better understand what to build next.

SECTION: company/strategy.md

Current strategy

Current strategy is in the internal handbook. The rest of this page describes the strategy process.

Three Year Outlook

Our strategy is looking 3 years out and fits into our cadences. It also maps to and reflects GitLab's mission, vision and values. It is a living strategy, so we will revisit on a regular basis and iterate when there is reason to do so.

Along the road to realizing our mission of everyone can contribute, our strategic goal is to be the leading complete DevSecOps Platform. We help our customers build

better software faster including developing cloud native applications, building mobile applications, and incorporating machine learning and artificial intelligence into their products and solutions. We believe that our dual flywheels (single application + open-core model) will propel us to efficiently be the default choice in the Enterprise market.

As we execute on our strategy, it is important to use our financial targets as guide rails and mature our internal processes for building a durable business for the long-term.

Principles

- 1. Fast follower: we don't have to be first to market and we don't suffer from Not Invented Here (NIH)
- 1. Values: make decisions based on our values, even if it is inconvenient.
- 1. Reach: go for a broad reach, no focus on business verticals or certain programming languages.
- 1. Seed then nurture: plant seeds and nurture the product to build out depth.
- 1. Speed: ship every change in the next release to maximize responsiveness and learning.
- 1. Life balance: we want people to stay with us for a long time, so it is important to take time off and work on life balance; being all-remote is a large part of the solution.
- 1. Open Source Stewardship: be a good steward of GitLab and collaborate with the wider community to improve the product together.

Seed then nurture

Our GitLab product vision is to deliver a single application for the entire DevOps lifecycle. This is a very expansive product vision that requires us to build an enormous amount of product surface area. Because we want to be efficient with capital, we want to build GitLab as a community. When we are early in a particular area of the product, we will plant seeds by shipping a small MVC. Shipping functionality that is incomplete to expand the scope sometimes goes against our instincts. However, planting those seeds even in an incomplete state allows others to see our path and contribute. With others contributing, we'll iterate faster and will accelerate the maturity of our offering faster than GitLab could on its own. We can have a long tail of categories that are at a minimal maturity that don't get investment until they show traction. While these come with a low level of shame they allow the wider community to contribute and people to express interest. It is much more common for people to contribute to categories that already exist and express interest in categories already shipping in the product. A minimal category is the placeholder to channel energy, and it is our responsibility to till the earth with minimal iterations.

GitLab the product should eventually have depth in every category it offers. To build out real depth requires a shift in focus, from planting seeds to nurturing the product area to maturity. We should concentrate our nurture investments in the categories that have demonstrated monthly active usage, revenue contribution, and demonstrated customer demand. As a product team, we'll be particularly focused on driving monthly active usage at the stage and group level. This should lead to more Stages per Organization, which is important as each stage added triples paid conversion! We'll also be heavily focused on driving usability, by measuring our system usability score, which is a measure of the user perception of GitLab's usability.

If we effectively seed and then nurture, we can fully activate GitLab's two growth turbos, by creating wider community contributions and driving more stages per user.

Assumptions

1. Open source user benefits: significant advantages over proprietary software because of its faster innovation, higher quality, freedom from vendor lock-in, greater security, and lower total cost of ownership.
1. Open Source stewardship: the wider community comes first, we play well with others and share the pie with other organizations commercializing GitLab.
1. Innersourcing is needed and companies will choose one solution top-down.
1. A single application where interdependence creates exceptional value is superior to a collection of tools or a network of tools. Even so, good integrations are important for network effects and making it possible to integrate GitLab into an organization.
1. To be sustainable we need an open core model that includes a proprietary GitLab EE.

Pricing

Please see our pricing model for details

Dual flywheels

GitLab has two flywheel strategies that reinforce each other: our open core flywheel and our development spend flywheel.

A flywheel strategy is defined as one that has positive feedback loops that build momentum, increasing the payoff of incremental effort.

You can visualize how the flywheels work in congruence via the diagram below. The KPI and responsibilities table lists the relevant indicator and department for every part of the flywheel.

In the open core flywheel, more features drive more users which in turn drive more revenue and more contributions which lead to more users.

The driving force behind the flywheel is that by using a DevOps platform to replace multiple point solutions, GitLab customers can achieve cost saving and efficiency gain. Therefore, when GitLab develops more features to improve the product maturity, it becomes easier to replace point solutions and GitLab will attract more users.

mermaid

graph BT;

```
id1(More Users)-->id2(More Revenue);
id2(More Revenue)-->id3(More Features);
id3(More Features)-.->id1(MoreUsers);
id1(More Users)-.->id4(More Contributions);
id3(More Features)-->id1(More Users);
id4(More Contributions)-.->id3(More Features);
```

KPIs and Responsible departments

Part of flywheel	Key Performance Indicator (KPI)	Department
-----	-----	-----
More Users	Stage Monthly Active Users	Product

More Contributions	Unique Wider Community Contributors per month	Developer Relations and Engineering
More Contributions	MRARR	Developer Relations and Engineering
More Contributions	Wider community contributions per release	Developer Relations and Engineering
More Features	Merge Requests per release per engineer in product development	Engineering and Product Management
More Revenue	IACV vs. plan	Sales and Marketing

Flywheel with two turbos

GitLab is a complete DevOps platform, delivered as a single application, with contributions from the wider community.

Compared to other DevOps platforms, GitLab leverages one or two unique turbos that boost the company:

1. Advantages of a single application, leading to more Stages per Organization
1. Open Source stewardship, leading to Wider community contributions

The advantage of a single application manifests itself if people start using more stages of the application.

In the graph below this is visualized with Stages per Organization (SpO), knowing that an organization using an extra stage triples conversion.

Increasing SpO drives both more seats and higher revenue per seat.

In the development spend flywheel, we capture the relationship between merge requests (MRs), changes in ARR from one period to the next (Delta ARR), hyper growth R&D spend and the resulting impact on MRs. We see that more MRs increase stage maturity which drives more monthly active users and stages per user which in turn drives more seats and more revenue which funds R&D spend and leads to more MRs.

mermaid

graph TB

```
id2(R&D Wider MR Rate) --> id1
id1(Wider community contributions Turbo) --> id3
style id1 color:ffffff, fill:9B51E0
id3(Stage maturity increase) --> id4
id3 --> id9
id4(MAU increase) --> id5
id5(More Licensed users) --> id6
id6(Net ARR) --> id12
id12(ARR) --> id8
id7(R&D spend ratio) --> id8
id8(R&D investment) --> id11
id11(Development Department Narrow MR Rate) --> id2
id9(SpO increase Turbo)--> id5
style id9 color:ffffff, fill:9B51E0
id9 --> id10
id10(Higher Revenue per licensed user) --> id6
```


id6 --> id7

Legend with links to the relevant metrics:

- 1. Wider community contributions Turbo
- 1. R&D Overall MR Rate
- 1. Stage maturity increase
- 1. MAU increase
- 1. SpO increase Turbo which is only possible in a single application with multiple stages
- 1. More Licensed users is due to an increase in unlicensed users (MAU) and the increase in SpO leading to a higher conversion from free to paid and a higher gross retention.
- 1. Higher Revenue per licensed user due to the Advantages of a single application
- 1. Net ARR stands for Annual Recurring Revenue is our most important KPI
- 1. ARR stands for Annual Recurring Revenue and IACV increases it.
- 1. R&D spend ratio becomes higher if the growth rate is higher due to the Hypergrowth Rule
- 1. R&D investment is the amount of money spend on Product Management and Engineering excluding Support
- 1. Development Department Narrow MR Rate

Publicly viewable OKRs and KPIs

To make sure our goals are clearly defined and aligned throughout the organization, we make use of Objectives and Key Results (OKRs) and Key Performance Indicators (KPIs) which are both publicly viewable.

North Star KPI

Our North Star KPI is revenue vs. plan. While our revenue data is limited access given that it is material nonpublic information (MNPI), revenue versus plan is a leading indicator of our likelihood of success in achieving revenue targets. Predictable revenue attainment is critical for continued business success.

Plan

Our yearly plans are linked from the relevant cadence section.

How will the three year strategy process work

- 1. The first phase of the strategy process is the Market Vision, SWOT and Positioning Strategy.
 - 1. Market Vision is our shared view of the market over the next ~3 years. Where is the market headed? What factors are influencing the market direction? Where are the points of change, as these will be the points of control?
 - 1. SWOT stands for Strengths, Weaknesses, Opportunities and Threats.
 - 1. Positioning strategy will highlight the way we want to position the company over the next three years. This will drive key decisions and tradeoffs we need to make in the strategy.
- 1. The second phase of the strategy process is taking the SWOT, positioning strategy and define the go-to-market (GTM), Product and Operations strategy. The output of these three strategies will be the strategic priorities for each, capabilities needed that we have and that we need to develop and the things we will explicitly not do. In addition we will define 3-5 big bets the

company will take - this will fold in under the strategic themes. Once we have these 3 to 5 themes we will synthesize them into a corporate strategy that ensures alignment among GTM, Product and Operations strategy.

1. Third phase will be the Long Range Plan process.

1. Fourth phase will be defining yearlies and the annual operating plan.

Big Bet Process

Activity

Positioning and SWOT published in internal handbook with description of strategy process two weeks post offsite

- May 15th e-group review (Complete)

E-group capability gap exercise at e-group offsite (prework)

- Stella

- Complete: Jun 28th

Complete capability gap exercise at VP level to bring more input into the group

- Directs Group

- Drafts: July 12th

- Final: July 19th

Pull together comprehensive view of capabilities from e-group and VP sessions
Craig/Dave

- July 24th

E-group June Virtual Offsite Sessions:

- Decide on GTM, Ops and Product strategies

- Decide on top capabilities for investment (feed into LRO)

- Craig/Dave

- Aug 1-3rd

Kickoff LRO with inputs from this process

SECTION: company/structure.md

Introduction

On this page we plan to take you through the organizational structure, layers and Job Frameworks that build GitLab as a company. We are working with two Job Frameworks, one for Individual Contributors and one for People Managers. We aim to explain how we use those frameworks, other projects and further considerations in GitLab's Organizational Structure.

Organizational chart

You can see who reports to whom in the most up-to-date organization chart by logging into Workday, choosing the Profile section, and select Team from the left sidebar to view Org Chart. Our HRIS is the Single Source of Truth for team member and organizational information.

Layers

Level (People Manager/IC)	Example(s)
Scope of impact	Expected Behaviors
----- -----	----- -----
----- ----- -----	
CEO	Chief Executive Officer
GitLab Global Organization	Champions
Executive	Chief People Officer and Chief Technology Officer Division
Champions	
VP/Fellow	VP of Global Channels
Department(s)	Drives Change
Senior Director	Senior Director, Engineering
Sub-department(s)	Develops the framework and strategy
Director/ Distinguished	Director of Customer Success Operations
Sub-department/multiple teams	Drives the framework, strategy and plans
Senior Manager/Principal	Principal Engineer
Across Sub-departments	Fosters
Manager/Staff	Engineering Manager
Across Teams	Implements
Senior	Senior People Connect Specialist
Cross functional work	Models
Intermediate	Intermediate Backend Engineer
Work within team	Grows/Acts
Associate	Junior Data Analyst
Own work	Learns/Develops

GitLab has at most eight layers in the company structure (Associate/Intermediate/Senior, Manager, Senior Manager, Director, Senior Director and/or VP, Executives, CEO). You can skip layers but you generally never have someone reporting to the same layer (Example of a VP reporting to a VP).

Dual Career Path at GitLab

A dual career path is a career path that allows upward mobility for team members without requiring that they be placed into a People Manager position. This structure serves as a way to advance team members who have deep specialist/technical skills but who are either 1) not interested or inclined to pursue a People Manager position and rather work as an Individual Contributor (IC), or 2) there is not sufficient business need to add additional layers of people management into a team at this time.

While today the dual-career path is most built out in our Engineering division, other divisions across the company have begun to build these out as well where structure and business need permits. The fork between specialist/technical work and the People Manager track is generally

above the Senior level. Once someone reaches a Senior-level role, and wants to progress, they will need to decide whether they want to remain purely specialist/technical or pursue a People Manager role. Their manager can provide opportunities to try tasks from both tracks if they wish. In most cases, the IC track and the people management path compensation bands align. Team members can review compensation details in our compensation calculator.

Job Frameworks

We've developed Job frameworks to provide clarity and consistency in our expectations for each job level at GitLab. There are two Job Frameworks: one for Individual Contributors and one for People Managers which are available to internal GitLab team members only, that map to the levels outlined in our job grades. If we make any updates to the frameworks we will always communicate that broadly. If you as a team member want to propose any changes we recommend that you to open a confidential issue in the People Business Partner project.

Goals of the Job Frameworks

The Job frameworks will help team members understand the career level at which they're contributing and understand the skills required for higher levels within the organization. Team members will also more easily see what skills are required of them at the next level in order to be ready to progress, when the business need supports the additional scope.

This guidance will better equip managers to have more productive conversations about performance expectations and career progression, enabl